Tursolite

A Rust, OCaml, OxCaml and Python library for parallel SQLite compatible databases



Alexy Khrabrov



FIRST PAIR
PRESS

Tursolite

A Rust, OCaml, OxCaml and Python library for parallel SQLite-compatible databases

Alexy Khrabrov, with Claude as co-author

2026-11-10

Version 0.2.0-df1cad42
Source commit df1cad429f8624ff91be1c46f8f9a00a59fb1553

First Pair Press
firstpair.press

Contents

About this manual 3
What tursolite is 4
  Where it fits 4
  What it is not 5
Getting started 6
  Building 6
  A first program 6
Engines, journals and sealing 8
  The engine 8
  The journal 8
  Sealing 8
Opening a file 9
  Options 9
  Reading without writing 9
  Choosing an engine 9
Connections, statements and transactions 10
  Threads and domains 10
  Transactions 10
  Statements and rows 10
  Values 10
Errors 11
The rules 12
Experimental features 13
Python 14
Testing 15
Limits 16
Reference 17
  Rust 17
  OCaml and OxCaml 17
  Python 17

About this manual

This manual is by Alexy Khrabrov, with Claude as co-author. The ideas and decisions are Alexy's; Claude wrote the text and much of the code with him.

It describes firstpair-tursolite 0.2.0. The source is querygraph/tursolite, under the Apache License 2.0. Turso's C header, `ocaml/sdk/turso.h`, is Turso's own, under the MIT license.

Two kinds of statement appear below, and they are kept apart. **Rules** are behaviours the library enforces, each found by testing Turso against stock SQLite. **Limits** are things it does not do, or that the pinned Turso does not do yet.

What tursolite is

A program that wants an embedded SQL file usually reaches for SQLite. Turso is a rewrite of SQLite in Rust that keeps its file format and adds what SQLite lacks, above all MVCC: several connections writing at once, each in its own transaction, with conflicts detected at commit.

The two engines look alike from far away, and differ in ways that matter up close. Turso has no rollback journal. A file it writes in MVCC mode is one stock SQLite refuses to open. Opening a standard file with Turso for writing changes its header. Each language reaches each engine differently too: Rust through crates, OCaml through the opam `sqlite3` package and a C library, Python through the standard `sqlite3` module and `pyturso`.

firstpair-tursolite puts one interface over both engines, with the same shape in four languages:

	SQLite	Turso	Turso MVCC, concurrent writers
Rust	rusqlite, bundled	the `turso` crate	threads, a connection each
OxCaml 5.4.0+ox	the opam `sqlite3`	the C ABI	domains, a connection each
Stock OCaml 5	the opam `sqlite3`	the C ABI	domains, a connection each
Python 3.12+	the standard `sqlite3`	`pyturso`	threads, a connection each

A program written against it chooses an engine with one option, and the library keeps the file one that both engines, and stock `sqlite3`, can read when it is closed.

Where it fits

This book is one of nine on the FirstPair photo tools. Each owns one part, and The FirstPair Photo Stack shows how they fit together.

- **tiffgraph** walks a TIFF or DNG file's structure under a fixed grammar and finds its editable XMP packet.
- **phidentify** computes a photo's identities: SHA-256, DNG raw blocks, image data, block marks, Lightroom fingerprint, metadata record and preserved digest, and the skeleton that rebuilds a file from an edited copy.
- **phindex** keeps those identities and a skeleton for every file of a volume, aligns two volumes by content, and restores a file as it was before a metadata edit.
- **DNG Compare** compares pairs and preserves one side byte for byte as deltas against the other, restorable even after the other side is edited, with a searchable metadata section, through `dngc`.
- **Semantic Unison** synchronizes photo trees, sending metadata edits without resending image bytes, keeps the versions it replaces as small skeletons, and preserves what a shuttle disk brings into the same archives.
- **packdir** keeps directories of small files as SQLite archives that still work like plain files.
- **Tursolite** is the SQLite and Turso layer under packdir, DNG Compare and Mechanic.
- **diskproxy** serves a folder as a virtual disk, such as previews with camera metadata spliced in, and captures ratings into sidecars.
- **Lightroom Repair** repairs Lightroom Classic catalogs, previews and Sync without forging their state.

Who uses it

tursolite is the database layer under three FirstPair projects. Each uses it for files that must outlive the program that wrote them:

- **packdir** keeps directories of small files as SQLite archives (`.sqlar`). Under Turso's MVCC its workers write one container at once, and `close` seals each container so stock `sqlite3 -Ax` still extracts it.
- **DNG Compare** keeps its preservation records (`recovery.sqlite3`), comparison caches and searchable metadata index in SQLite files. Its archives stay in WAL between runs, which is what `Keep_wal` and `Journal::Existing` are for, and it reads closed archives with `immutable`.
- **Mechanic**, which builds Photo Mechanic catalogs from Lightroom previews, keeps its EXIF stub store in one SQLite file through it, from Rust and from OxCaml. It writes the store under Turso's MVCC, one connection per thread or domain, and the finished store is served read-only with `immutable` by diskproxy, which is why sealing back to a rollback journal with no `-wal` beside it matters.

What it is not

tursolite is not an ORM, a migration tool or a server. It runs SQL you write, on a file you name. It does not hide which engine opened a file: `engine` says which one did, and a program that cares can ask.

Getting started

Building

```
cargo build --features turso        # Rust: features sqlite (default) and turso
make sdk                            # Turso's C library, for OCaml
opam pin add firstpair-tursolite-sdk .
opam pin add firstpair-tursolite .
make test                           # Rust, stock OCaml, OxCaml 5.4.0+ox and Python
```

Turso is pinned exactly, at 0.8.3, in every language. The opam package `firstpair-tursolite-sdk` holds Turso's C library, built by `make sdk` outside opam because opam's sandbox allows no `cargo`. Without the SDK package, `firstpair-tursolite` builds on SQLite alone and refuses Turso when asked, with the error kind `Refused`.

The library built by `make sdk` carries a stamp, a copy of `Cargo.lock`, and a build copies the library only when the stamp matches. An out-of-date Turso cannot be linked by accident.

The OCaml tests run in two opam switches: `default`, with stock OCaml 5, and `5.4.0+ox`, with OxCaml. Pin both packages in each switch that uses them.

A first program

The same program in three languages: open a file with Turso in MVCC mode, create a table, insert a row in a transaction, close.

```rust
use firstpair_tursolite::{Database, Engine, Journal, Options, Value};

let o = Options::new().engine(Engine::Turso).journal(Journal::Mvcc);
let db = Database::open("x.db".as_ref(), &o)?;
let c = db.connect()?;                      // one per thread
c.exec("CREATE TABLE IF NOT EXISTS t(a)")?;
c.transaction(|| c.run("INSERT INTO t VALUES (?)", &[Value::Integer(1)]))?;
drop(c);
db.close()?;                                // seals what Turso wrote
```

```
open Tursolite

let db = open_database ~engine:Turso ~journal:Mvcc "x.db" in
let c = connect db in                       (* one per domain *)
exec c "CREATE TABLE IF NOT EXISTS t(a)";
transaction c (fun () -> run c "INSERT INTO t VALUES (?)" [Integer 1L]);
disconnect c;
close db                                    (* seals what Turso wrote *)
```

```
from firstpair_tursolite import Database, Engine, Journal

db = Database.open("x.db", engine=Engine.TURSO, journal=Journal.MVCC)
c = db.connect()                            # one per thread
```

```
c.exec("CREATE TABLE IF NOT EXISTS t(a)")
c.transaction(lambda: c.run("INSERT INTO t VALUES (?)", [1]))
c.close()
db.close()                                  # seals what Turso wrote
```

After `close`, `sqlite3 x.db 'SELECT * FROM t'` prints `1`. The file is a standard SQLite database again.

Engines, journals and sealing

The engine

`engine` is `Sqlite` (the default) or `Turso`. The engine that actually opened a file is reported by `engine` on the database, because the library may choose differently from what was asked when the file requires it (see *Choosing an engine* below).

The journal

`journal` says how the file keeps its transactions:

Journal	Meaning	Engines
`Delete` (default)	SQLite's rollback journal	SQLite; with Turso, the file is sealed back to it on close
`Wal`	write-ahead log	both
`Mvcc`	Turso's multi-version concurrency: concurrent writers	Turso only; asked of SQLite it is refused
`Existing`	leave the file's journal as it is found	both

`Existing` is for programs that manage their own files, such as DNG Compare's archives, which stay in WAL between runs.

Sealing

Turso and SQLite disagree about what a file looks like while it is in use. So `close` *seals* what Turso wrote: it checkpoints, leaves MVCC, checkpoints again, and then, as the `seal` option says:

- **`Standard`** (the default) hands the file to SQLite to return it to a rollback journal, as SQLite itself would have made it, where SQLite is linked.
- **`Keep_wal`** (`KeepWal` in Rust, `KEEP_WAL` in Python) leaves it a WAL file, which Turso reopens as it was.

Empty `-wal` and `.db-log` files are removed. One with content is an error, because it may hold commits. A crash leaves a file unsealed until Turso next opens it.

Close every connection before `close`.

Opening a file

Options

Every option has a default, so a program names only what it changes:

Option	Default	Meaning
`engine`	`Sqlite`	which engine to ask for
`journal`	`Delete`	see above
`page_size`	none	set on a new file only
`auto_vacuum`	false	incremental auto-vacuum on a new file; SQLite only
`read_only`	false	open without writing
`immutable`	false	SQLite's `immutable=1` (below)
`create`	true	create the file when missing
`busy_ms`	60000	how long a writer waits for another to finish
`seal`	`Standard`	what `close` leaves
`experimental`	none	Turso's experimental features (below)

Options grow without breaking callers. Rust's `Options` is `#[non_exhaustive]` and built from `Options::new()` with one method per field. OCaml and Python take every option by name with its default.

Reading without writing

`read_only` opens a file without writing it.

`immutable` is SQLite's `immutable=1`, for a file that nothing changes while it is open: no locks, no journal, and nothing written beside the file. It implies `read_only`, always uses SQLite, and refuses an unsealed MVCC file. It is the right way to read a closed archive on a volume another program may also be reading, because a plain read-only open of a WAL file still creates `-wal` and `-shm` files beside it.

Choosing an engine

`inspect` reads the first 100 bytes of a file and returns its header: page size, the two version bytes, and whether it has auto-vacuum. It returns nothing for a missing or empty file, and the error `Not_a_database` for anything that is not SQLite.

`resolve` decides which engine may open a file for what is wanted, and `open_database` applies it:

- A file in MVCC mode opens only with Turso, whatever was asked.
- A file with auto-vacuum is never written by Turso. Asked to read it with Turso, the library reads it with SQLite instead; asked to write it, it refuses.
- Otherwise the engine asked for is used, if this build links it.

Connections, statements and transactions

Threads and domains

A database is shared. A connection belongs to one thread (Rust, Python) or one domain (OCaml) at a time, and each takes its own from `connect`. The first `connect` hands out the connection that set the file up.

Under MVCC every connection may write at once. Otherwise writers take turns, each waiting up to `busy_ms` for the one before.

Transactions

`transaction` runs a function inside `BEGIN IMMEDIATE`, or `BEGIN CONCURRENT` under MVCC, and commits when it returns. Under MVCC a transaction that loses a conflict to another writer rolls back and runs again. **The function must therefore be repeatable**: it should compute what it writes from what it reads inside the transaction, not from state outside it that the first attempt changed.

A row that every writer updates, such as a counter or a summary, belongs in a short transaction of its own, so that conflicts on it are cheap to retry.

Statements and rows

`exec` runs SQL with no parameters, `run` one statement with parameters, and `query` returns rows as arrays of values in column order. The rest are conveniences over these:

	Rust	OCaml	Python
All rows, with column names	`query_rows` (`Rows { columns, rows }`)	`query_rows` (`{ columns; rows }`), `column`	`query_rows` (`Rows(columns, rows)`), `.column`
One row at a time	`for_each`, `query_map`, with `Row::named`	`iter`, `fold`	`rows`, `for_each`
The first row	`query_one`	`query_one`	`query_one`
A single value	`query_value::<T>`	`query_value`	`query_value`
The last rowid	`last_insert_rowid`	`last_insert_rowid`	`last_insert_rowid`

Each connection prepares a statement once and reuses it: rusqlite's and Turso's caches in Rust, a per-connection cache over the opam `sqlite3` in OCaml, the standard module's own cache in Python. The OCaml Turso C ABI and `pyturso` prepare a statement each time.

Values

A value is `Null`, `Integer` (64-bit), `Real`, `Text` or `Blob`, in every language. Rust converts with `From` and `FromValue` (including `Option<T>` for nullable columns) and `as_i64`, `as_f64`, `as_str`, `as_bytes`. OCaml has `as_int64`, `as_int`, `as_float`, `as_string`, `is_null` and `as_option`. Python uses `None`, `int`, `float`, `str` and `bytes`.

Errors

Every failure is one error with a kind to match on, and a message:

Kind	Meaning	What to do
`Conflict`	busy, locked, or a concurrent transaction that lost	roll back and retry; `transaction` does
`Constraint`	a constraint failed	a problem in the data
`Not_a_database`	the file is not SQLite, or is MVCC and SQLite was used	check the file and the engine
`Read_only`	a write on a read-only open	
`Refused`	one of the rules below, or an engine this build does not link	read the message
`Other`	anything else	

Rust has `Error { kind, message }` with `ErrorKind`. OCaml raises `exception Error of { kind; message }`, with `error_kind_name` for messages. Python raises `Error` with `.kind` (`ErrorKind.CONFLICT` and so on) and `.message`.

The rules

Each rule came from testing Turso against stock SQLite, on Turso 0.8.2 and 0.8.3, and is enforced when a file is opened:

- **Turso never writes a file with auto-vacuum.** With its experimental flag on, stock SQLite's integrity check then fails on the file. Reads fall to SQLite, and a write is an error.
- **A file in MVCC mode opens only with Turso.** Its header bytes 18 and 19 are 255, and SQLite calls it "not a database".
- **Turso has no rollback journal.** It ignores `journal_mode = DELETE`, which is why `close` seals.
- **Opening a standard file with Turso for writing changes its header.** A read-only open does not.
- **Foreign keys are off on every SQLite connection.** The bundled SQLite in Rust turns them on by default and the system's does not, so one program behaved differently in two languages until the library set one rule. A program that wants them says so with `PRAGMA foreign_keys = ON`.

Experimental features

`experimental` turns on Turso's experimental features, none by default. Asking for one with the SQLite engine is refused.

The one offered is `Without_rowid` (`WithoutRowid`, `WITHOUT_ROWID`), for `CREATE TABLE … WITHOUT ROWID`, which Turso 0.8.3 refuses without it. With it, Turso writes such a table, and a file sealed `Standard` passes SQLite's integrity check. Turso 0.8.3 still refuses inserts into one in MVCC mode, will not create a secondary index on one, and will not write one that SQLite made with such an index. A table SQLite made without such an index opens, reads and takes writes with no flag, in WAL mode.

Python

```
cd python
uv sync                                  # pyturso==0.8.3 and pytest
uv run pytest -q                         # or: make test-python
pip install './python[turso]'            # elsewhere; without [turso] it is SQLite
only
```

The package is `firstpair_tursolite`, one module over the standard `sqlite3` and `pyturso`. Python takes Turso from the `turso` extra, `pyturso==0.8.3`, and refuses any other version.

`pyturso` 0.8.3 differs from the Rust and C interfaces in three ways the package works around:

- **No read-only open**, and opening a standard file rewrites its header. So a read-only open of any file not in MVCC mode goes to SQLite, which reads it untouched; `engine` says so.
- **No busy timeout.** Statements that start a transaction, or stand alone, wait out `busy_ms` in the package instead.
- **No statement cache.**

Testing

`make test` runs one matrix of tests in every language: `tests/matrix.rs`, `ocaml/test/matrix.ml` under both opam switches, and `python/tests/test_matrix.py`. They cover each engine and journal, sealing, the rules, concurrent writers and the error kinds.

The Python tests include a check across languages: a file Python's Turso path wrote and sealed is read by the Rust crate's SQLite engine, through `cargo test`. A sealed file is meant to be readable by any engine in any language, and that test holds the library to it.

Limits

- **Turso is pinned at 0.8.3.** Its experimental features, and the rules above, are re-tested before the pin moves.
- **`Without_rowid` tables cannot take inserts under MVCC** in Turso 0.8.3, even with the experimental flag. A program that wants both writes an ordinary rowid table under Turso, then rebuilds it `WITHOUT ROWID` with SQLite once the concurrent writing is done; Mechanic's stub store is made that way.
- **The OCaml Turso C ABI and `pyturso` prepare every statement**, which costs time on statement-heavy work.
- **An unsealed file**, left by a crash, stays in Turso's form until Turso opens it again. SQLite, and `immutable`, refuse it in MVCC mode.

Reference

Rust

```
Database::open(path, &Options::new().engine(..).journal(..)..) -> Result<Database>
db.engine() -> Engine           db.mvcc() -> bool
db.connect() -> Result<Connection>
db.close() -> Result<()>
c.exec(sql)   c.run(sql, &[Value])   c.query(sql, &[Value])
c.query_rows  c.for_each  c.query_map  c.query_one  c.query_value::<T>
c.last_insert_rowid()
c.transaction(|| ..)
inspect(path) -> Result<Option<Header>>
resolve(path, Engine, read_only) -> Result<Engine>
Engine::{Sqlite, Turso}   Journal::{Delete, Wal, Mvcc, Existing}
Seal::{Standard, KeepWal}  Experimental::WithoutRowid
Value::{Null, Integer, Real, Text, Blob}
Error { kind: ErrorKind, message }
```

OCaml and OxCaml

```
val open_database : ?engine:engine -> ?journal:journal -> ... -> string -> database
val engine : database -> engine          val mvcc : database -> bool
val connect : database -> connection     val disconnect : connection -> unit
val close : database -> unit
val exec : connection -> string -> unit
val run : connection -> string -> value list -> unit
val query : connection -> string -> value list -> value array list
val query_rows : connection -> string -> value list -> rows
val iter, fold, query_one, query_value, last_insert_rowid
val transaction : connection -> (unit -> 'a) -> 'a
val inspect : string -> header option
val resolve : string -> engine -> read_only:bool -> engine
type engine = Sqlite | Turso
type journal = Delete | Wal | Mvcc | Existing
type seal = Standard | Keep_wal
exception Error of { kind : error_kind; message : string }
```

`database` and `connection` are abstract; the full interface is `ocaml/lib/tursolite.mli`.

Python

```
Database.open(path, Options(...))  or  Database.open(path, engine=...,
journal=..., ...)
db.connect()  db.close()  db.engine
c.exec  c.run  c.query  c.query_rows  c.rows  c.for_each  c.query_one  c.query_value
c.last_insert_rowid
c.transaction(fn)  c.close()
```

```
Engine.SQLITE/TURSO  Journal.DELETE/WAL/MVCC/EXISTING  Seal.STANDARD/KEEP_WAL
Experimental.WITHOUT_ROWID  Error(.kind, .message)  ErrorKind.CONFLICT ...
```